MAPÚA MALAYAN DIGITAL COLLEGE
of Mapúa Malayan Colleges Laguna
In collaboration with
Arizona State University®

Lost and Found Reporting System (FindMe)

Prepared and Presented by:

Menadel Miguel
Mikee Miguel
Jay-R Dagami
Sean Plaza

S3101 | Group 13
Bachelor of Science in Information Technology
Term 1 SY. 2026-2027

Intellectual Property Notice

This template is an exclusive property of **Mapua-Malayan Digital College** and is protected under **Republic Act No. 8293**, also known as the Intellectual Property Code of the Philippines (IP Code). It is provided solely for educational purposes within this course. Students may use this template to complete their tasks but may not **modify, distribute, sell, upload,** or **claim ownership** of the template itself. Such actions constitute copyright infringement under **Sections 172, 177, and 216** of the IP Code and may result in legal consequences. Unauthorized use beyond this course may result in legal or academic consequences.

Additionally, students must comply with the **Mapua-Malayan Digital College Student Handbook**, particularly with the following provisions:

- **Offenses Related to MMDC IT**:
    - **Section 6.2** – Unauthorized copying of files
    - **Section 6.8** – Extraction of protected, copyrighted, and/or confidential information by electronic means using MMDC IT infrastructure
- **Offenses Related to MMDC Admin, IT, and Operations**:
    - **Section 4.5** – Unauthorized collection or extraction of money, checks, or other instruments of monetary equivalent in connection with matters pertaining to MMDC

Violations of these policies may result in **disciplinary actions ranging from suspension to dismissal**, in accordance with the Student Handbook.

For permissions or inquiries, please contact MMDC-ISD at isd@mmdc.mcl.edu.ph.

Table of Contents

TABLE OF CONTENTS..... 3

I. Project Documentation..... 4

A. Project Information..... 4

B. Project Repository Information..... 5

II. Backend-Supported Features Analysis..... 6

III. Completed Implementation Work..... 8

IV. Implementation Dependencies..... 9

A. Key dependencies..... 11

V. Workflow Updates and Revisions..... 11

VI. Deployment..... 13

VII. App Development Workflow..... 13

VIII. Development Reflections..... 15

A. What worked well..... 15

B. What needs improvement..... 16

IX. Team Discussion Outcomes..... 16

X. Testing Evidence..... 18

I. Project Documentation

A. Project Information

Item	Details
Project Title	Lost and Found Reporting System (FindMe)
Project Type	Web-Based Information System
Project Purpose	To provide a centralized platform for reporting, searching, and managing lost and found persons, pets, and items.
Target Users	General Users and System Administrators
Main Problem Addressed	Lost-and-found information is often scattered across social media posts, group chats, and other platforms, making reports difficult to organize and find.
Main Goal	Make lost-and-found information easier to submit, search, view, and manage.
Development Approach	Object-Oriented Systems Analysis and Design (OOSAD)

The Lost and Found Reporting System (FindMe) is designed to provide one centralized platform where users can report lost or found persons, pets, and items. Users can provide important information such as descriptions, dates, locations, photos, and contact details. Administrators can manage submitted reports and help maintain organized and appropriate information.

B. Project Repository Information

The project repository contains the documents, diagrams, designs, and other files needed for the development of the Lost and Found Reporting System.

Github:

https://github.com/mgmiguel24/webssystemsandtechnology-Group13-lostandfoundreportingsystem

Repository Content	Description
Project Proposal	Background, problem statement, objectives, proposed solution, and scope
Milestone 1 Documentation	Interactive Frontend Web Application documentation (DOCX and PDF)
Backend Application (findme-api)	Milestone 2 Express.js API, SQLite database, web interface, and automated tests
Live Prototype (Milestone 1)	Milestone 1 prototype
Live System (Milestone 2)	Deployed backend web application on Render
Class Diagram	Main classes, attributes, methods, and relationships of the system
Component Diagram	Major software components and how they interact
UI/Prototype Design	Planned interface screens and user interactions

The repository is updated through reviewed pull requests, so the main branch always holds the latest working version.

II. Backend-Supported Features Analysis

The Milestone 1 prototype simulated backend behavior in the browser using localStorage. In Milestone 2, we moved these features to an Express.js server with a SQLite database, so data is shared across users and every permission is enforced on the server.

Feature	Milestone 1 Prototype	Milestone 2 Implementation
User registration	Saves user information in localStorage	Registration API; passwords hashed with bcrypt
Login and logout	Checks credentials in the browser	Server-side authentication with a secure session cookie that expires after 24 hours
User roles	Role selected during registration	Roles assigned by the server; sign-up always creates a regular user
Lost and found reports	Saves reports locally	Stored in a SQLite database through the API

Search and filters	Filters reports in the browser	Server-side search with filters for type, category, status, and a date range
Report editing	Not available in the prototype	Owner or admin can edit a report; every edit is logged in its history
Report status	Owner or admin can change status in the prototype	Server verifies ownership or admin role before any change
Delete report	Admin-only interface check	Admin-only, enforced on the server; deleted reports are kept for the audit trail
Photo upload	Stores images as Base64 data	Saved as files on the server; JPEG, PNG, WebP, or GIF up to 5 MB
Contact information	Visible in the report detail modal	Hidden from guests; shown only to signed-in users
Dark mode	Theme toggle, not saved	Theme choice saved in the browser and restored on the next visit

Moving these checks to the server closed the security gaps in the prototype, where any permission could be bypassed by editing browser data. Authentication, role authorization, privacy, and data storage are now handled by the backend.

III. Completed Implementation Work

The following features are implemented in the current system.

- Responsive interface with a card-based report layout.
- Light and dark theme toggle, with the choice saved between visits.
- User registration, login, and logout handled by the server.
- Header displaying the logged-in user's name and role.
- Lost and found report submission for people, pets, and items, with an optional photo.
- Report details including location, date, description, contact information, and case history.
- Keyword search with filters for type, category, status, and a date range.
- Report editing by the report owner or an administrator.
- Mark as Resolved and Reopen actions, with a confirmation step, for report owners or administrators.
- Delete action restricted to administrators on the server; deleted reports are kept for the audit trail.
- Contact details hidden from guests and shown only to signed-in users.
- Administrator tools to list users and change roles, with protection against removing the last administrator.
- Rate limiting on login attempts and report submissions.
- An administrator account and 5 sample reports created automatically for testing.

Current status: The frontend and backend are both implemented and connected. Authentication, role-based authorization, and report management are handled server-side via the Express.js API. The application has been tested with an automated test suite and deployed publicly on Render.

IV. Implementation Dependencies

The following components are used in the current system.

Component	Purpose	Status
HTML, CSS, JavaScript	Build the browser interface and interactions	Implemented
Tailwind CSS	Style the responsive interface	Implemented
Phosphor Icons	Provide interface icons	Implemented
Browser localStorage	Remember the user's theme choice	Replaced by the API for all data; now only remembers the theme

Node.js and Express.js	Build backend services and REST API routes	Implemented
Database (PostgreSQL or SQLite)	Persistent storage for users and reports	Implemented
bcryptjs	Password hashing	Implemented
jsonwebtoken + httpOnly cookies	Session authentication	Implemented
multer	Photo upload handling, with file-type validation	Implemented
zod	Request validation	Implemented
helmet, express-rate-limit	Security headers and rate limiting on auth and report creation	Implemented
node:test (automated test suite)	Verifies auth, report, and admin behavior end-to-end	Implemented

Render (hosting)	Public deployment	Implemented — free tier, see limitations below

A. Key dependencies

- Frontend and backend are fully connected; all `fetch()` calls in the UI point to live API endpoints.
- Database schema is finalized and in use (users, reports, report_events).
- Authentication and role-based authorization are enforced on every protected route.
- Photo uploads are validated and stored as real files, not embedded as Base64.

V. Workflow Updates and Revisions

We followed a prototype-first workflow: build the interface, test the user journey, identify problems, and resolve them during backend integration. The table records each issue found in Milestone 1 and how we resolved it in Milestone 2.

Area	Issue identified	Resolution	Status
Authentication	Passwords are stored in plain text	Passwords are hashed with bcrypt, and login runs on the server	Resolved
User roles	Users can select the admin role during registration	Role selection removed; sign-up always creates a regular user, and only an admin can promote users	Resolved

Contact privacy	Contact details are visible to visitors	Contact details are hidden from guests and shown only to signed-in users	Resolved
Image storage	Base64 images can exceed storage limits	Photos are saved as files on the server, limited to 5 MB and image formats only	Resolved
Dark mode	Theme may not apply consistently	Theme class moved to the page root so dark mode applies to the whole page	Resolved
Theme persistence	Theme selection is lost after reload	Theme choice is saved and restored on the next visit	Resolved
Search and filters	Date filtering supports only an exact date	Added From and To date range filters	Resolved
Report management	No report editing or confirmation before status changes	Added an Edit Report form for owners and admins, and a confirmation before status changes	Resolved

Input validation	Future report dates are accepted	Future dates are blocked in the form and rejected by the server	Resolved
Interface consistency	Different system names appear in the interface	Header, footer, and page title now consistently use FindMe	Resolved

VI. Deployment

The application is deployed on Render (free tier) and publicly accessible at:
https://find-me-tx2o.onrender.com

Known limitations of the current deployment:

- The free tier has no persistent disk. The SQLite database and uploaded photos reset to seeded sample data whenever the service restarts, redeploys, or wakes from sleep.
- The free tier sleeps after approximately 15 minutes of inactivity; the first request afterward can take 30–50 seconds to respond while the service wakes up.
- These are hosting-tier constraints, not application bugs. A paid tier with a persistent disk, or migration to a hosted database service, would resolve both issues for a production deployment.

VII. App Development Workflow

The backend was built in the following order, so that each stage could be tested before the next depended on it:

Phase	Work	Outcome
1. Setup & Schema	Set up the Express project structure (routes/, controllers/, models/, middleware/); design the database schema (users, reports, report_events); configure environment variables	Project skeleton and database schema in place
2. Authentication	Build registration, login, logout; implement password hashing and JWT session cookies; enforce that registration can never create an admin	Secure, working login system
3. Core Report Features	Build report creation, listing, search/filter, status change, and soft delete, each with server-enforced permissions	Reports can be created, found, and managed through the API
4. Photo Upload & Access Control	Add multer-based photo upload with file validation; apply requireAuth, requireRole, and ownerOrAdmin middleware across all protected routes	Uploads and permissions fully server-side, replacing the prototype's browser-only checks
5. Testing	Write an automated end-to-end test suite (http://api.test.js) covering guest, user, and admin flows	Confidence that core features work correctly before extending them
6. Enhancements	Add report editing, last-admin protection, date range filtering, and rate	Closed the remaining gaps identified in the

	limiting on report creation	Workflow Updates table
7. Deployment	Deploy to Render; connect GitHub for automatic redeploys on push; configure environment variables and seed data	Publicly accessible, working application
8. Frontend Integration	Replace all localStorage calls in the prototype with fetch() calls to the live API	Frontend and backend fully connected

VIII. Development Reflections

A. What worked well

- Automated testing before deployment. We wrote 22 automated tests covering sign-up, login, permissions, privacy, search, editing, and admin safeguards. Running them before each merge caught issues early, such as an unclear error message on an empty login, which we fixed before deploying.
- Pull request workflow. Every backend change went through a pull request that was reviewed before merging. This kept the main branch stable and gave us a clear record of who changed what.
- Layered backend structure. Splitting the backend into routes, controllers, models, and middleware made each feature easier to build and test separately, which fixed the single-file problem we noted in Milestone 1.
- Prototype-first foundation. The Milestone 1 interface gave us a working user journey to build on, so the backend could replace browser storage without redesigning the screens.

B. What needs improvement

- Deploying on Render's free tier showed that data resets when the server restarts, something local testing never revealed. A future version should use persistent storage or a hosted database.

IX. Team Discussion Outcomes

Discussion topic	Decision/ Outcome
Backend technology	We built the backend with Node.js and Express.js to handle the REST API, authentication, and report management.
Database	We selected SQLite to store user accounts, reports, and a history log of every report action.
Authentication	Registration, login, and logout run on the server. Passwords are hashed with bcrypt and never stored as plain text. Sessions use a secure cookie that expires after 24 hours. Login attempts and report submissions are rate limited to block password guessing and spam.
User permissions	The admin account is created from server settings, and the sign-up form ignores any request for admin access. Report owners and

	admins can edit a report and update its status, only admins can delete reports, and admins can promote other users. The system blocks removal of the last remaining admin.
Report management	We finalized report creation, search, filtering, editing, status updates, and deletion. Search supports a date range, and future dates are rejected. Every edit is recorded in the report's history, and deleted reports are hidden rather than erased, so admins keep a record.
Privacy	Contact details are hidden from guests and shown only to signed-in users. Report lists never include contact information.
Photo uploads	Photos must be JPEG, PNG, WebP, or GIF and no larger than 5 MB. The server checks the actual file contents, not just the file name, and saves each photo under a random name.
Testing	We wrote 22 automated tests covering security, sign-up, login, permissions, privacy, search and date filters, report editing, admin safeguards, rate limiting, and deletion. We also tested the live deployment across guest, user, and admin flows.
Deployment	The system is deployed on Render's free tier. We accepted that data and uploaded photos may reset on server restart, since the free plan has no permanent storage.

Version control	Changes go through pull requests and are reviewed before merging into the main branch. Merged branches are deleted to keep the repository clean.
Task assignments	**Jay-R Dagami:** Backend development, documentation and deployment. **Sean Plaza:** README documentation. **Mikee Miguel:** Pull request review and merging, README documentation, and final documentation **Menadel Miguel:** Frontend design prototype and documentation

X. Testing Evidence

We tested the live deployment at https://find-me-tx2o.onrender.com across guest, regular user, and administrator flows. The screenshots below show each core feature working on the deployed system.

Demo login

Role	Email	Password
Admin	admin@findme.local	SystemAdminGroup13

Regular user accounts can be created through the Register Account form.

1. **Guest view: reports listed, contact details hidden.**

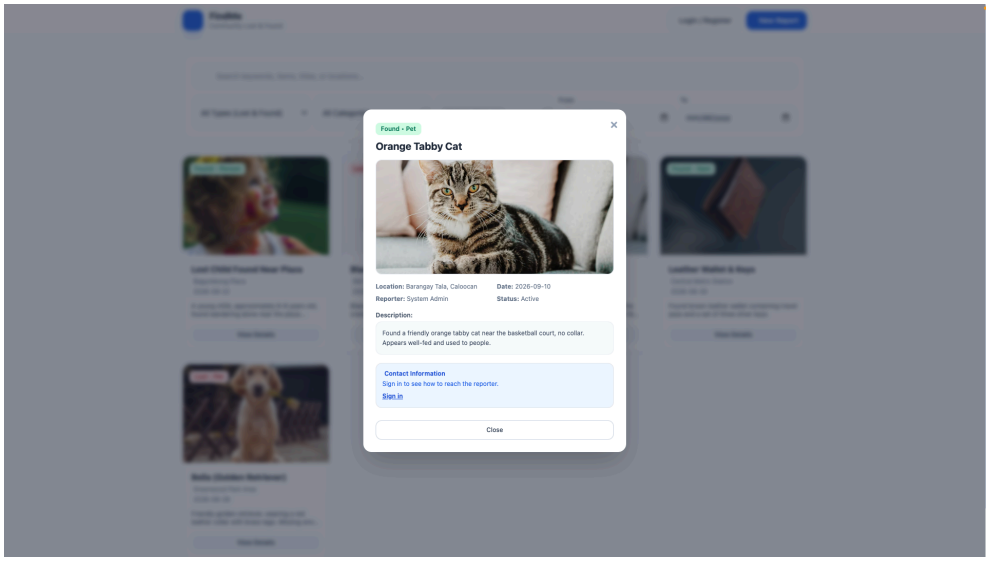


Figure1:

https://github.com/mgmiguel24/webssystemsandtechnology-Group13-lostandfoundreportingsystem/blob/main/Testing%20Evidence/Figure%201%20.png

2. **Registration of a new regular user account.**

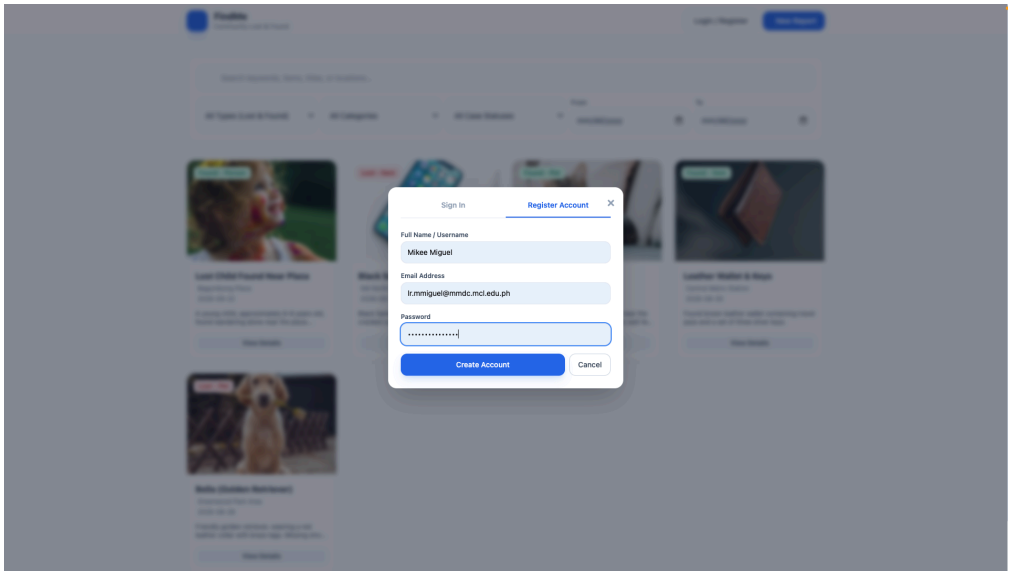


Figure 2:

https://github.com/mgmiguel24/webssystemsandtechnology-Group13-lostandfoundreportingsystem/blob/main/Testing%20Evidence/Figure%202.png

3. **Signed-in user filing a new report with a photo.**

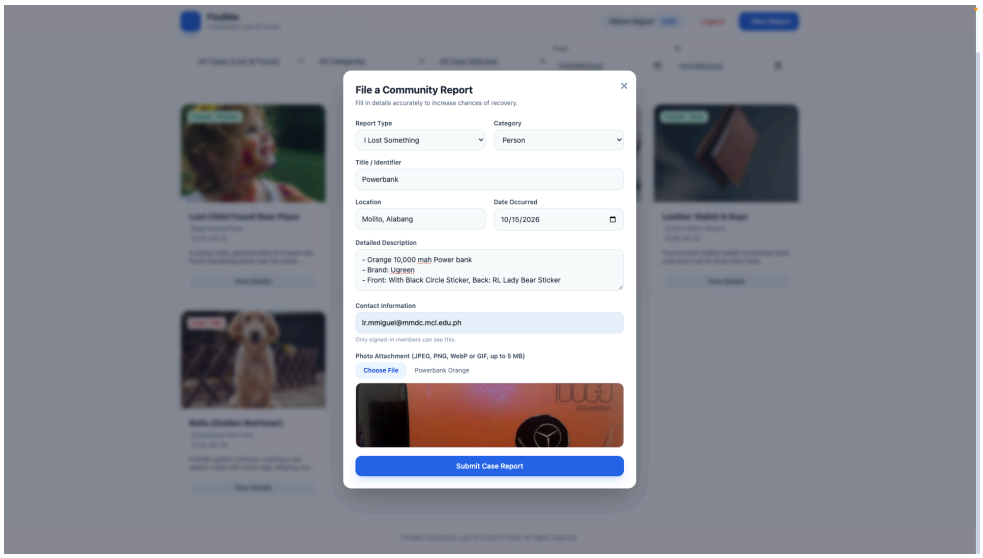


Figure 3:

https://github.com/mgmiguel24/webssystemsandtechnology-Group13-lostandfoundreportingsystem/blob/main/Testing%20Evidence/Figure%203.png

4. **Search with the From and To date range filter applied.**

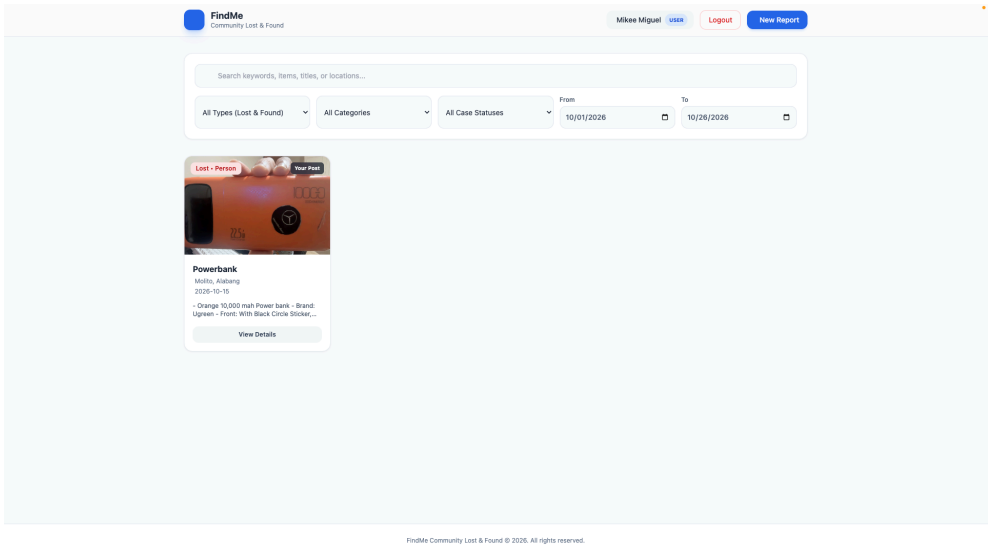


Figure 4:

https://github.com/mgmiguel24/webssystemsandtechnology-Group13-lostandfoundreportingsystem/blob/main/Testing%20Evidence/Figure%204.png

5. **Report owner editing a report.**

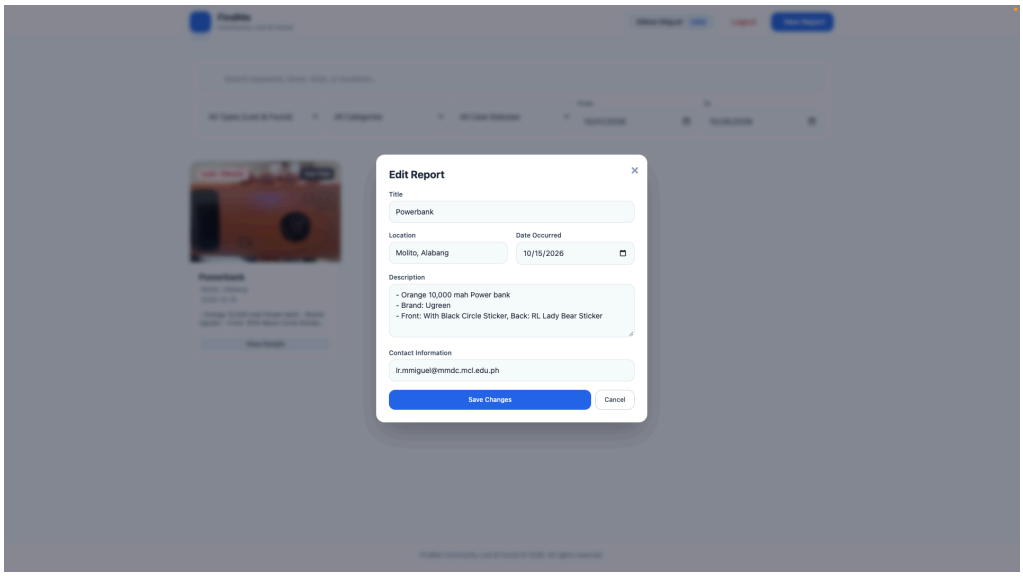


Figure 5:

https://github.com/mgmiguel24/webssystemsandtechnology-Group13-lostandfoundreportingsystem/blob/main/Testing%20Evidence/Figure%205.png

6. **Report marked as Resolved, with the update shown in its case history.**

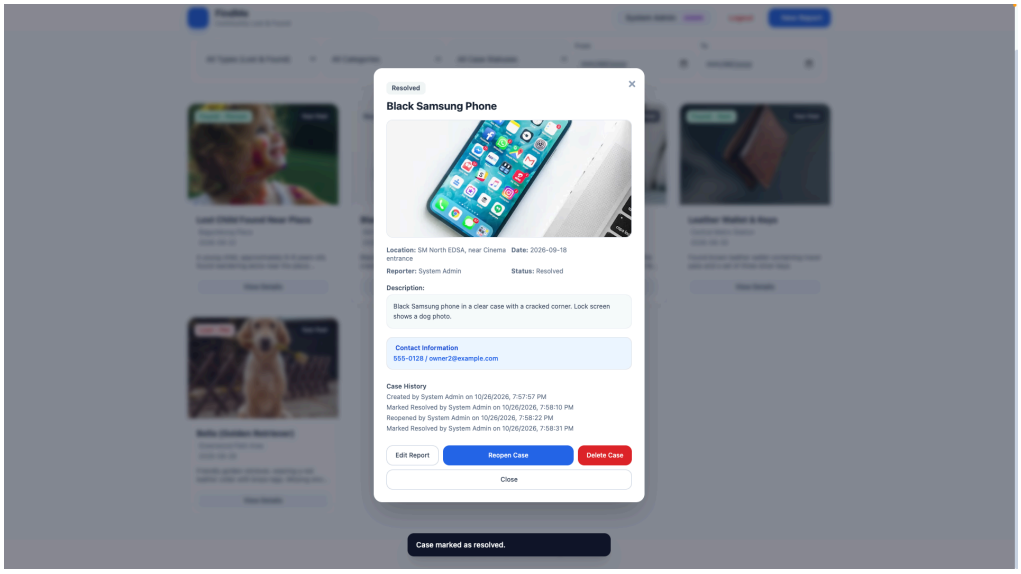


Figure 6:

https://github.com/mgmiguel24/webssystemsandtechnology-Group13-lostandfoundreportingsystem/blob/main/Testing%20Evidence/Figure%206.png

7. **Admin deleting a report, with the confirmation prompt.**

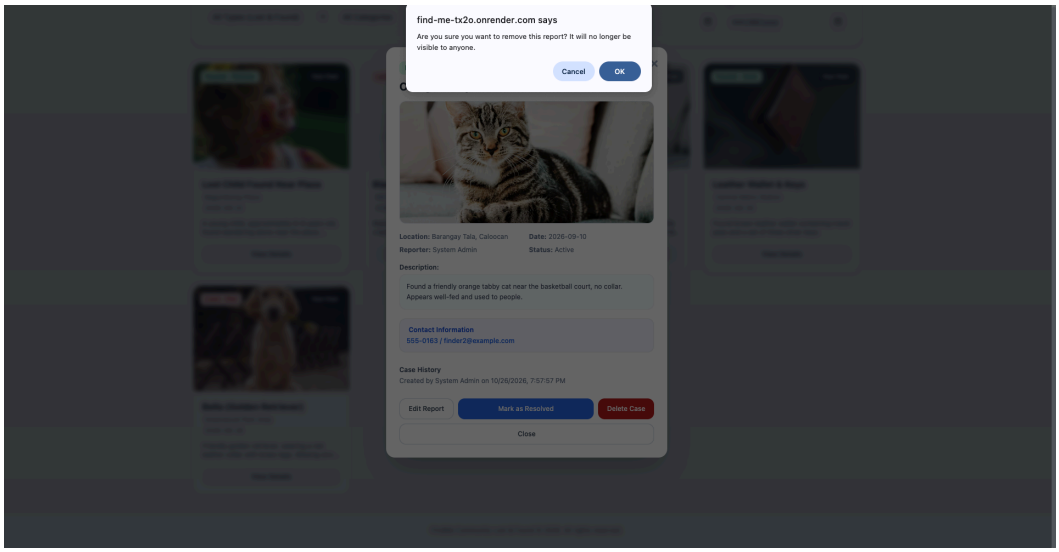


Figure 7:

https://github.com/mgmiguel24/webssystemsandtechnology-Group13-lostandfoundreportingsystem/blob/main/Testing%20Evidence/Figure%207.png

8. **Automated test results from npm test**

```
09:24:42 PM  ==> Downloading cache...
09:24:42 PM  ==> Cloning from https://github.com/mgmiguel24/webssystemsandtechnology-Group13-lostandfoundreportingsystem
09:24:48 PM  ==> Checking out commit 80620604f1c224e3ab7a53b2dcf0b1e0c5e2d9d1 in branch main
09:24:48 PM  ==> Downloaded and extracted 40 MiB cache in 6s.
09:24:49 PM  ==> Requesting Node.js version >=20
09:24:49 PM  ==> Using Node.js version 26.12.1 via /opt/render/project/src/findme-api/package.json
09:24:49 PM  ==> Docs on specifying a Node.js version: https://render.com/docs/node-version
09:24:49 PM  ==> Installing Node.js version 26.12.1...
09:24:50 PM  ==> Running build command 'npm install && npm test'...
09:24:51 PM
09:24:51 PM  changed 2 packages, and audited 98 packages in 438ms
09:24:51 PM
09:24:51 PM  34 packages are looking for funding
09:24:51 PM    run `npm fund` for details
09:24:51 PM
09:24:51 PM  found 0 vulnerabilities
09:24:51 PM
09:24:51 PM  > findme-api@1.0.0 test
09:24:51 PM  > node --test tests/api.test.js
09:24:51 PM
09:24:51 PM  [seed] Admin account created: admin@test.local
09:24:52 PM  ✓ security headers are sent (48.224581ms)
09:24:52 PM  ✓ registration ignores a requested admin role (634.148975ms)
09:24:52 PM  ✓ duplicate email and weak password are rejected (6.094693ms)
09:24:53 PM  ✓ wrong password returns 401; correct one signs in (609.075994ms)
09:24:53 PM  ✓ passwords are stored hashed, never as plain text (0.682436ms)
09:24:53 PM  ✓ guests cannot create reports (3.629142ms)
09:24:53 PM  ✓ signed-in user creates a report with a photo (29.158335ms)
09:24:53 PM  ✓ invalid reports are rejected (14.063478ms)
09:24:53 PM  ✓ contact details are hidden from guests and shown to members (4.245448ms)
09:24:53 PM  ✓ search and filters run on the server (4.99269ms)
09:24:53 PM  ✓ date range filters include only reports within the requested bounds (1.5772ms)
09:24:53 PM  ✓ dateFrom and dateTo filters work independently (2.975911ms)
09:24:53 PM  ✓ exact date filter remains supported (1.456028ms)
09:24:53 PM  ✓ exact and range date filters reject invalid dates (3.457182ms)
```

Figure 8.1:

https://github.com/mgmiguel24/webssystemsandtechnology-Group13-lostandfoundreportingsystem/commit/87d04541c967f2635c8fd347de0d90c9c6774232

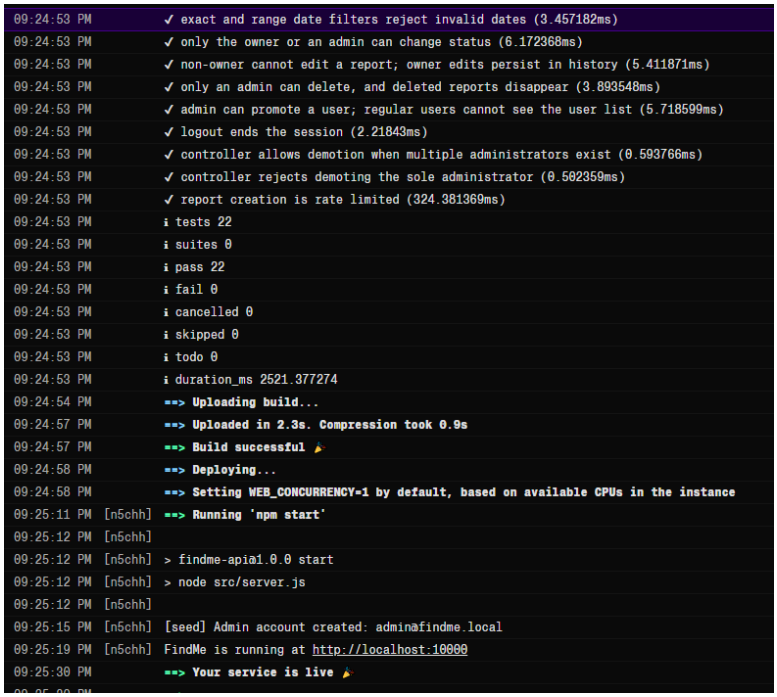


Figure 8.2:

https://github.com/mgmiguel24/webssystemsandtechnology-Group13-lostandfoundreportingsystem/blob/main/Testing%20Evidence/Figure%208.2